

# AI Audit (Hash Chain) — Technical Reference

**Audience:** Engineers working on Fyn AI, audit, or compliance code paths. **Sprint:** S0.12 (Sprint 0). **Spec invariants:** INV-2.10.2 (hash-chain audit), INV-2.5.4 (audit trail matches reality). **Browser acceptance:** BS-15 ([tests/Browser/scenarios/BS-15-hash-chain-audit-admin-view.php](#)).

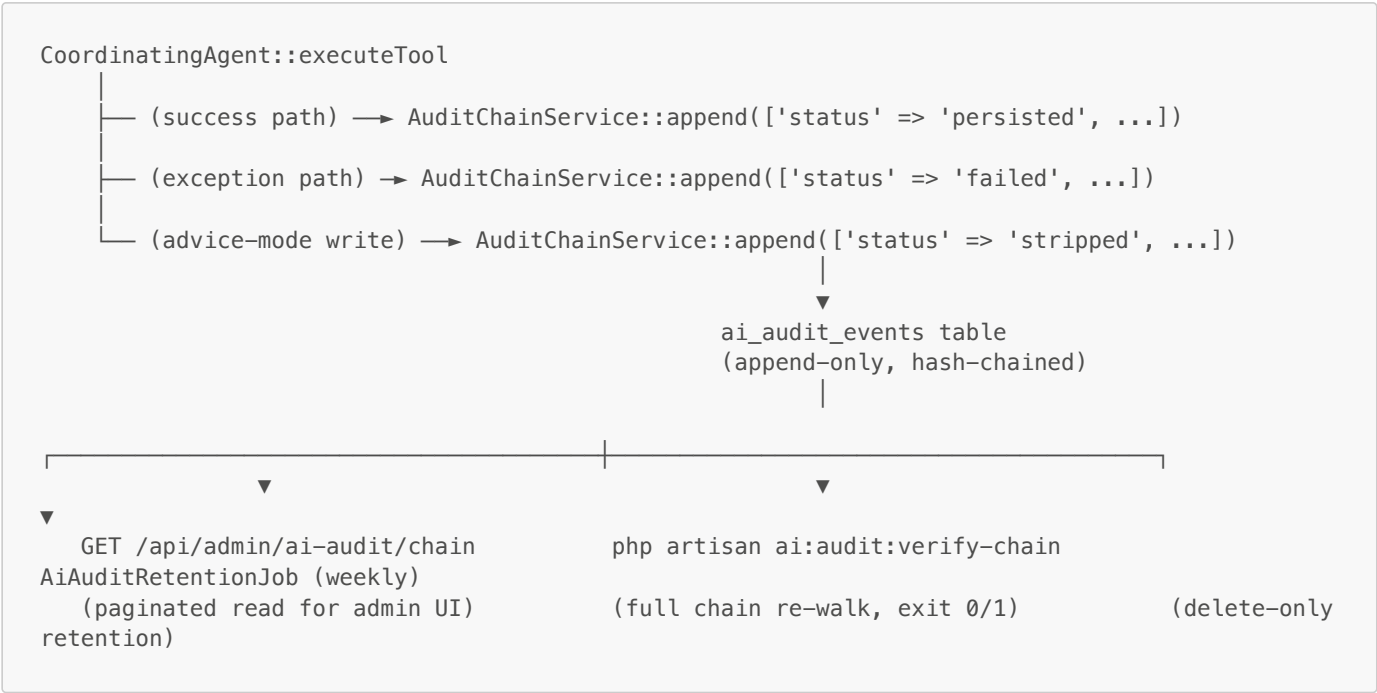
## What this replaces

Pre-S0.12, AI tool calls were recorded as text lines in `storage/logs/laravel.log` written from `CoordinatingAgent::executeTool ([AI-AUDIT] tool=... user=... ...)`. The file channel had three problems:

- 1. **Mutable.** Anyone with write access to `storage/logs/` could edit, truncate, or fabricate lines.
- 2. **Unverifiable.** No way to prove a historical line was the line originally written.
- 3. **Unstructured.** Filtering required ad-hoc grep; cross-referencing to a conversation or entity required parsing free-form text.

S0.12 replaces this with an append-only, hash-chained, HMAC-signed MySQL table that satisfies INV-2.10.2 and is queryable via SQL and a JSON API.

## Architecture overview



Every AI-mediated tool call lands as one row. Every row carries the cryptographic fingerprint of every prior row. The chain is verified weekly by cron and on-demand from the admin UI.

## Database schema

**Migration:** `database/migrations/2026_04_25_000013_create_ai_audit_events_table.php` **Model:** `app/Models/AiAuditEvent.php` **Table:** `ai_audit_events`

| Column          | Type                                      | Notes                                   |
|-----------------|-------------------------------------------|-----------------------------------------|
| id              | bigIncrements                             | Primary key, ascending = chronological. |
| user_id         | foreignId → users.id                      | cascadeOnDelete.                        |
| conversation_id | foreignId → ai_conversations.id, nullable | nullOnDelete.                           |

| Column         | Type                                               | Notes                                                                                                       |
|----------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| tool_name      | string(64)                                         | e.g. <code>create_savings_account</code> , <code>get_recommendations</code> .                               |
| operation      | enum('read','write','handoff','classify')          | The semantic class of tool call.                                                                            |
| status         | enum('dispatched','persisted','failed','stripped') | Lifecycle state at the moment of append.                                                                    |
| input_summary  | json nullable                                      | Tool input, summarised. Cast to PHP array.                                                                  |
| result_summary | json nullable                                      | Tool result or exception summary. Cast to PHP array.                                                        |
| entity_type    | string(32) nullable                                | e.g. <code>SavingsAccount</code> . Set on <code>persisted</code> writes.                                    |
| entity_id      | unsignedBigInteger nullable                        | The id of the touched entity.                                                                               |
| prev_hash      | char(64)                                           | SHA-256 hex of the previous row. Zero-hash for the first row.                                               |
| row_hash       | char(64)                                           | SHA-256 hex of <code>prev_hash</code> + <code>canonical_json(payload)</code> + <code>signed_at_iso</code> . |
| signed_at      | timestamp                                          | The exact moment of append. Hashed in.                                                                      |
| signature      | char(64)                                           | HMAC-SHA256 of <code>row_hash</code> keyed on <code>config('app.ai_audit_hmac_key')</code> .                |
| created_at     | timestamp default CURRENT_TIMESTAMP                | Equals <code>signed_at</code> for new rows.                                                                 |

Indexes:

- `(user_id, created_at)` — primary admin query path: "all events for user X over time".
- `(tool_name, status)` — secondary path: "all failed `create_savings_account` calls".
- `row_hash` — used by `verifyChain()` if walking by hash rather than id.

**Timestamps disabled on the model** (`public $timestamps = false;`). The migration manages `created_at` directly so it never differs from `signed_at`.

## The hash chain

**Service:** `app/Services/AI/AuditChainService.php`

Fields covered by the row hash

```
private const HASHED_FIELDS = [  
    'user_id',  
    'conversation_id',  
    'tool_name',  
    'operation',  
    'status',  
    'input_summary',  
    'result_summary',  
    'entity_type',  
    'entity_id',  
];
```

The chain columns themselves (`prev_hash`, `row_hash`, `signature`, `signed_at`, `created_at`, `id`) are intentionally **not** covered by `row_hash` — including them would make the hash self-referential. `signed_at` is fed into the hash separately as a string suffix.

## Computing `row_hash`

```
row_hash = sha256(prev_hash · canonical_json(payload) · signed_at_iso)
```

Where:

- `prev_hash` = the 64-char hex `row_hash` of the immediately preceding row, or 64 zero-bytes (`0000...0000`) for the first row.
- `canonical_json(payload)` = `json_encode` with `JSON_THROW_ON_ERROR | JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE` after **canonicalising** the payload.
- `signed_at_iso` = `signed_at->toIso8601String()`.

## Canonical JSON — the MySQL JSON column trap

This is critical and the cause of the only production bug fixed during BS-15 verification.

MySQL's binary `JSON` column type **reorders object keys on storage** — keys are sorted by length ascending, ties by insertion order. The write-time hash uses the PHP array's iteration order; the read-back-and-verify hash uses MySQL's reordered cast-back. For any payload where the PHP order doesn't already match MySQL's sort, the two hashes diverge and `verifyChain()` reports `broken_at`.

The fix: `computeRowHash` calls `canonicaliseForHash()` first, which recursively `ksort`s every associative array (numeric lists preserve order). Both write-time and verify-time produce the same canonical bytes, regardless of:

- PHP array iteration order at append time.
- MySQL's internal JSON key reordering.
- Driver round-trip casting differences.

```
private static function canonicaliseForHash(mixed $value): mixed
{
    if (! is_array($value)) {
        return $value;
    }

    $isList = array_is_list($value);
    $result = [];
    foreach ($value as $k => $v) {
        $result[$k] = self::canonicaliseForHash($v);
    }

    if (! $isList) {
        ksort($result);
    }

    return $result;
}
```

The pre-fix Pest test happened to use `['index', 'preview']` — lengths 5 and 7, which matched MySQL's sort order — so the bug was masked from the unit tests. The fix is covered by a regression test in `tests/Feature/Audit/HashChainTest.php` ("verifies chains whose input\_summary keys MySQL would reorder") with payloads `['preview', 'q']` and `['preview', 'q', 'provider', 'sum_assured', 'policy_type']`.

## Append, atomically

```
public function append(array $event): AiAuditEvent
{
    return DB::transaction(function () use ($event) {
        $prev = AiAuditEvent::query()->lockForUpdate()->latest('id')->first();
        $prevHash = $prev?->row_hash ?? self::ZERO_HASH;
```

```

    $signedAt = now();
    $payload = [/* HASHED_FIELDS extracted from $event */];

    $rowHash = self::computeRowHash($prevHash, $payload, $signedAt->toIso8601String());
    $signature = hash_hmac('sha256', $rowHash, (string) config('app.ai_audit_hmac_key'));

    return AiAuditEvent::create([...$payload, 'prev_hash' => $prevHash, 'row_hash' =>
    $rowHash, 'signed_at' => $signedAt, 'signature' => $signature, 'created_at' => $signedAt]);
  });
}

```

**Concurrency:** the `lockForUpdate()` on the latest row serialises concurrent writers. Only one append can determine "the previous row" at a time, so the chain stays single-threaded under load. The transaction guarantees that if the insert fails, no half-chained row is left behind.

### Verifying the chain

```

public function verifyChain(): array
{
    $prevHash = self::ZERO_HASH;
    $count = 0;

    foreach (AiAuditEvent::query()->orderBy('id')->cursor() as $row) {
        $payload = self::extractHashedPayload($row);
        $expected = self::computeRowHash($prevHash, $payload, $row->signed_at-
        >toIso8601String());

        if (! hash_equals($expected, (string) $row->row_hash)) {
            return [
                'chain_valid' => false,
                'broken_at' => (int) $row->id,
                'row_count' => $count,
            ];
        }

        $prevHash = $row->row_hash;
        $count++;
    }

    return [
        'chain_valid' => true,
        'tip_hash' => $prevHash,
        'row_count' => $count,
    ];
}

```

- Walks the table in `id` order via `cursor()` (streams rows — does not load the whole table into memory).
- Re-derives every `row_hash` from the previous one using the same `computeRowHash` the appender used.
- Returns `{chain_valid: true, tip_hash, row_count}` on success.
- Returns `{chain_valid: false, broken_at, row_count}` on the first mismatch (where `broken_at` is the id of the first bad row, and `row_count` is how many good rows preceded it).
- Uses `hash_equals` for constant-time comparison — paranoid but cheap.

### Verifying the HMAC signature alone

```

public function verifySignature(AiAuditEvent $row): bool
{
    $expected = hash_hmac('sha256', (string) $row->row_hash, (string)
    config('app.ai_audit_hmac_key'));
}

```

```
    return hash_equals($expected, (string) $row->signature);
}
```

Used when you want to check that a specific row was signed by the holder of the current `AI_AUDIT_HMAC_KEY` — e.g. for forgery detection independent of chain integrity. Not currently exposed via API.

## Configuration

**Key:** `config/app.php` exposes `ai_audit_hmac_key`, sourced from the `AI_AUDIT_HMAC_KEY` environment variable.

```
# .env (production)
AI_AUDIT_HMAC_KEY=<64-byte hex string, generated once, never rotated lightly>
```

**Generating a key** (one-time, per environment):

```
php -r "echo bin2hex(random_bytes(32));"
```

**Rotation policy:** rotating the key invalidates every existing row's signature. If rotation is genuinely required (compromise suspected), do it via a planned migration that re-signs every row using the old chain to derive new signatures — never silently swap the env var. Production and dev each have their own key; the dev key is in `deploy/cs-jones-fynla/.env.production`, the production key in `deploy/fynla-org/.env.production`.

## API surface

All endpoints under `routes/api.php` middleware: `auth:sanctum` + admin guard (the prefix `admin/ai-audit` sits inside the admin route group).

### GET /api/admin/ai-audit/users

List users with at least one AI conversation. Used by the **Conversations** tab user list.

- Query: `search` (optional, matches email/first\_name/surname), `page`.
- Returns paginated `{id, name, email, is_preview_user, conversation_count, last_conversation_at}`.

### GET /api/admin/ai-audit/users/{userId}/conversations

Conversations for one user. Returns `{user, conversations[]}` where each conversation includes `id, title, status, model_used, message_count, total_input_tokens, total_output_tokens, created_at, last_message_at`.

### GET /api/admin/ai-audit/conversations/{conversationId}/messages

Full message thread for one conversation, plus the latest `AiAdviceLog`. Used by the right pane.

### GET /api/admin/ai-audit/chain — S0.12

Paginated read of the hash-chain ledger.

- Query: `user_id, status, operation, page` (50 per page).
- Returns paginated rows with **all** chain fields exposed: `prev_hash, row_hash, signature, signed_at`, plus the standard metadata.
- Sort: `ORDER BY id DESC` (newest first).

### GET /api/admin/ai-audit/chain/verify — S0.12

Walk the entire chain and return the verifier result.

- No params.

- Returns `{success: true, data: {chain_valid: true, tip_hash, row_count}}` or `{success: true, data: {chain_valid: false, broken_at, row_count}}`.
- Same JSON shape as the artisan command.

## CLI

```
php artisan ai:audit:verify-chain
```

**Command:** `app/Console/Commands/AiAuditVerifyChainCommand.php`

- Walks the chain via `AuditChainService::verifyChain()`.
- Emits the JSON to stdout (one line, `JSON_UNESCAPED_SLASHES`).
- Exit 0 on `chain_valid: true`, exit 1 on `chain_valid: false`.
- Used by ops, the Sprint 0 verification rollup, and the weekly cron.

## Scheduled jobs

**Defined in:** `app/Console/Kernel.php`.

```
$schedule->job(new \App\Jobs\AiAuditRetentionJob())->weeklyOn(0, '04:00');
$schedule->command('ai:audit:verify-chain')->weeklyOn(0, '04:30');
```

- Sunday 04:00 — `AiAuditRetentionJob` prunes aged-out rows.
- Sunday 04:30 — `ai:audit:verify-chain` walks the post-prune chain. The exit code lets ops alerting catch a broken chain within a week.

## Retention windows

**Job:** `app/Jobs/AiAuditRetentionJob.php`

| Class           | Window                          | Selector                                                                       |
|-----------------|---------------------------------|--------------------------------------------------------------------------------|
| Long retention  | 2,555 days ( $\approx$ 7 years) | <code>operation = 'write' OR tool_name = 'get_recommendations'</code>          |
| Short retention | 730 days ( $\approx$ 2 years)   | everything else (read tools, classifier hits, handoff chrome, stripped events) |

**Long-retention rows** are the regulator-relevant trail of writes and personalised recommendations. **Short-retention rows** are operational chrome.

### Critical design choice — delete, do not pseudonymise.

The job deletes aged-out rows rather than mutating them. Mutating any historical row (e.g. swapping a name field for a pseudonym) would change the row's serialised payload, change its expected `row_hash`, and `verifyChain()` would correctly report a break. Deletion does not break the chain because `verifyChain()` walks from the earliest surviving id with the canonical zero-hash predecessor — anything not in the table is outside its remit.

**For GDPR pseudonymisation:** if/when needed, do it in a separate read-only export view that re-serialises the row with PII fields swapped — the source rows stay untouched and the chain stays verifiable. That export view is **out of scope for Sprint 0**.

## Frontend

**Component:** `resources/js/components/Admin/AiAudit.vue` **Service:** `resources/js/services/aiAuditService.js`

**Mounted under:** `resources/js/views/Admin/AdminPanel.vue` (Admin → AI → AI Audit).

### Component state

```
{
  activeTab: 'conversations', // or 'chain'
  // Conversations tab
  users, conversations, messages, adviceLog,
  selectedUser, selectedConversation, searchQuery,
  loadingUsers, loadingConversations, loadingMessages,
  expandedSections,
  // Chain tab
  chainEvents, chainLoading,
  chainFilters: { userId, status, operation },
  chainPagination: { current_page, last_page, total },
  chainStatus: { loading, result },
}
```

### Lazy load on tab switch

`onChainTabSelected()` only fetches the chain on the first switch into the tab — subsequent switches keep the cached page. **Re-verify** always hits the server.

### Status pill mapping

```
statusBadgeClass(status) {
  switch (status) {
    case 'persisted': return 'bg-spring-100 text-spring-700';
    case 'failed':    return 'bg-raspberry-100 text-raspberry-700';
    case 'stripped':  return 'bg-violet-100 text-violet-700';
    case 'dispatched':
    default:          return 'bg-horizon-100 text-horizon-700';
  }
}
```

All colours are Tailwind tokens from the v1.2.0 design system (no hex). The hash column shows `row.row_hash.slice(0, 12) + '...'`. The full hash is exposed only via `data-tip-hash` on the banner span and the `title` tooltip.

### Banner re-verify behaviour

Clicking **Re-verify** sets `chainStatus.loading = true`, calls `aiAuditService.verifyChain()`, and writes the result back to `chainStatus.result`. The chain table itself is not re-fetched — only the verification banner. Re-fetching the table is a separate path (`loadChain(page)` from filter changes or pagination).

## Adding a new audited tool call

When a new AI tool is added to `CoordinatingAgent`, audit it like this:

```
// At dispatch (entering the tool):
$this->auditChain->append([
  'user_id' => $userId,
  'conversation_id' => $conversationId,
  'tool_name' => 'create_protection_policy',
  'operation' => 'write',
  'status' => 'dispatched',
  'input_summary' => $sanitisedInput, // never raw PII; summarise
]);

try {
  $result = $this->doTheWork(...);

  // On success:
  $this->auditChain->append([
```

```
        'user_id' => $userId,
        'conversation_id' => $conversationId,
        'tool_name' => 'create_protection_policy',
        'operation' => 'write',
        'status' => 'persisted',
        'input_summary' => $sanitisedInput,
        'result_summary' => ['policy_id' => $result->id],
        'entity_type' => 'ProtectionPolicy',
        'entity_id' => $result->id,
    });
} catch (\Throwable $e) {
    // On failure:
    $this->auditChain->append([
        'user_id' => $userId,
        'conversation_id' => $conversationId,
        'tool_name' => 'create_protection_policy',
        'operation' => 'write',
        'status' => 'failed',
        'input_summary' => $sanitisedInput,
        'result_summary' => ['error' => $e->getMessage()],
    ]);
    throw $e;
}
```

For **advice-mode strip** events (write tools removed from the catalogue before the LLM saw them — see `AdviceFyn::WRITE_TOOLS`), append with `status: 'stripped'` and `operation: 'write'` so the audit row proves the read-only contract held.

For **classifier** audits (Sprint 1+), use `operation: 'classify'`. For **handoff** rows (advice → capture switch via `delegate_to_capture`), use `operation: 'handoff'`.

## Testing

| Test                                                          | What it covers                                                                                                                                                                                                                      |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tests/Feature/Audit/HashChainTest.php                         | Chain coherence under append; canonical-JSON regression for MySQL key reordering.                                                                                                                                                   |
| tests/Feature/Audit/ChainTamperDetectionTest.php              | Mutating a historical row reports <code>chain_valid: false, broken_at: &lt;id&gt;</code> .                                                                                                                                          |
| tests/Feature/Audit/RetentionPseudonymisationTest.php         | Retention job deletes the right classes; pseudonymisation does NOT happen in place; chain remains verifiable post-prune.                                                                                                            |
| tests/Browser/scenarios/BS-15-hash-chain-audit-admin-view.php | End-to-end Playwright walk: admin login → AI Audit → Chain view tab → banner shows valid + tip_hash → Re-verify works → artisan command tip_hash matches DOM tip_hash byte-for-byte. <b>Status: GREEN, session 92 (2026-04-26).</b> |

Run all audit tests:

```
./vendor/bin/pest tests/Feature/Audit/
```

Verify the chain end-to-end from the CLI (uses the live DB):

```
php artisan ai:audit:verify-chain
```



## Known limitations and future work

1. **No bulk export view.** Forensic exports currently require direct SQL. A signed export view (with re-serialised PII pseudonymisation that preserves chain verifiability via a parallel export hash) is out of scope for Sprint 0.
2. **HMAC key rotation is destructive.** Rotating `AI_AUDIT_HMAC_KEY` invalidates every existing signature. A planned re-sign migration would be needed for safe rotation.
3. **Conversations tab and Chain tab are not joined in the UI.** To go from a conversation to its audit rows, you have to switch tabs and filter manually. A "show audit rows for this conversation" link is a candidate for Sprint 1.
4. **No alerting wired to the weekly verifier.** The Sunday 04:30 cron exits 1 on a broken chain, but nothing reads that exit code yet. Hook it into the Sprint 1+ ops alerting.
5. **`verifySignature()` is not exposed via API.** Useful for forgery detection independent of chain integrity, but no current consumer.
6. **Conversation\_id column not displayed in the chain table.** It's stored, queryable, and returned by the API — just not rendered in the admin UI yet.

## File map

| Layer              | Path                                                                                |
|--------------------|-------------------------------------------------------------------------------------|
| Migration          | <code>database/migrations/2026_04_25_000013_create_ai_audit_events_table.php</code> |
| Model              | <code>app/Models/AiAuditEvent.php</code>                                            |
| Service            | <code>app/Services/AI/AuditChainService.php</code>                                  |
| Controller         | <code>app/Http/Controllers/Api/AiAuditController.php</code>                         |
| Routes             | <code>routes/api.php</code> (lines 1066–1073)                                       |
| Artisan command    | <code>app/Console/Commands/AiAuditVerifyChainCommand.php</code>                     |
| Retention job      | <code>app/Jobs/AiAuditRetentionJob.php</code>                                       |
| Schedule           | <code>app/Console/Kernel.php</code> (lines 40–41)                                   |
| Frontend component | <code>resources/js/components/Admin/AiAudit.vue</code>                              |
| Frontend service   | <code>resources/js/services/aiAuditService.js</code>                                |
| Pest — chain       | <code>tests/Feature/Audit/HashChainTest.php</code>                                  |
| Pest — tamper      | <code>tests/Feature/Audit/ChainTamperDetectionTest.php</code>                       |
| Pest — retention   | <code>tests/Feature/Audit/RetentionPseudonymisationTest.php</code>                  |
| Browser scenario   | <code>tests/Browser/scenarios/BS-15-hash-chain-audit-admin-view.php</code>          |

## Related references

- **Spec invariants:** `April/April24Updates/spec/01-invariants.md` — INV-2.10.2, INV-2.5.4.
- **Test strategy:** `April/April24Updates/spec/03-test-strategy.md` §BS-15.
- **Two-Fyn canonical contract:** `April/April24Updates/spec/00-canonical.md` — explains why `stripped` events exist.
- **User guide:** `docs/manuals/ai-audit-hash-chain-user-guide.md` — how admins read the view.